

Web Application Vulnerability Assessment

Tool Used: OWASP ZAP

Assessment Type: Dynamic Application Security Testing (DAST)

Target: <http://testphp.vulnweb.com>

Date: Feb. 22, 2026

Analyst: Jonathan Montalvo

1. Executive Summary

The tests were done in an authorized training environment using OWASP ZAP. These tests were done to identify typical web application security vulnerabilities, interpret the results to interpret the methodology used by SOC Analysts and application security testers.

2. Scan Overview

Scan Type	Duration	HTTP Requests	Total Alerts
Manual Explore + Active Scan	2m 15s	597	31
Automated Explore + Active Scan	10m 26s	7476	37

3. Key Findings

High Severity

Vulnerability	Count	Description	Impact
Cross Site Scripting	20	User-controlled data reflected in responses	Credential theft, session hijacking, malware

SQL Injection -MySQL	10	User input is embedded into queries without sanitization	Data extraction, authentication bypass
-------------------------	----	--	--

Medium Severity

Vulnerability	Count	Description	Impact
XSLT Injection	2	Malformed XML transformation tests triggered	Possible data exposure
GET-for-POST Misuse	3	Endpoints accept improper methods	Could bypass validation
http only site	2	The site is only served in http	Security, user trust, search engine rankings, and functionality
Directory	3	It is posible to view a list of directory content	May reveal scripts, files, backup source files, etc.

Low Severity

Vulnerability	Count	Description
Missing Anti-clickjacking Header	1	Increases risk of clickjacking
Private IP Disclosure	1	This info can be used for further atatchks

Informational

Vulnerability	Notes
User-Agent Fuzzing	Automated scan produced a large volume of inconsistent responses.

Parameter Tampering Tests	No confirmed vulnerabilities
---------------------------	------------------------------

4. Observations

- Majority of alerts consists of input validation attacks
- Backend Protection seems stronger

5. Recommendations

- **Implement Server-Side Validation:** Always validate inputs on the server, not just the client-side, using allow-lists (whitelists) for expected formats.
- **Parameterized Queries:** Use prepared statements for SQL queries to prevent command injection.
- Add essential security headers
- Disable inline JavaScript
- **Set cookie HttpOnly, Secure, and SameSite flags:** These cookie settings protect user sessions by preventing JavaScript access, blocking insecure transmission, and limiting cross-site cookie misuse.
- Implement HTTPS using TLS certificates